NAVUUNA

Backend Build Plan & System Architecture

Making Africa Investment Green

Codebase: nuvola-atlas-* · Companion to the Navuuna Build Phases tracker

Version 1.0 · July 12, 2026 · HEAD 92626c3 · Scope: code & infrastructure only

Contents

1. Purpose & Scope

This document is the engineering plan for building, hardening, and operating the Navuuna backend: the Laravel 11 core platform, the PostgreSQL/PostGIS data layer on Supabase, the FastAPI ingestion microservice, and the supporting infrastructure (Forge/DigitalOcean, Fly.io staging, Cloudflare, Sentry, Reverb). It consolidates every backend-facing task from the Build Phases tracker (Phases A–F) into a single blueprint with the system architecture, database tables, service layer, API surface, and security posture drawn out explicitly.

Authority chain — this document sits below the tracker. Where anything here conflicts with the Navuuna Build Phases file, the tracker wins; where the tracker conflicts with what ships in the codebase, the codebase wins and the deviation is logged. This file tracks code and infrastructure only — finance, people/culture, legal, and funding live in WorkToDo.md.

Critical Rules (carried over from the tracker)

- Read the Build Phases tracker before every session. Do not redo completed items ([x]).
- Do not skip incomplete items unless explicitly told to.
- Update the checkbox to [x] immediately when a task is completed.
- Never delete or overwrite the tracker — only update checkboxes, Last updated / HEAD lines, and append new phases.
- The repo's tasks/todo.md is the live tactical execution plan; the tracker is the authoritative phase-level record.

2. System Architecture

Navuuna is a spatial intelligence platform for Nairobi County. The backend is split into two deliberately isolated services so that a heavy batch of city data can never take the public-facing API down: the Laravel core platform (auth, scoring, spatial APIs, exports, realtime) and the FastAPI ingestion engine (validation, cleaning, anomaly detection). They communicate over a single hardened internal channel authenticated with X-Internal-Secret.

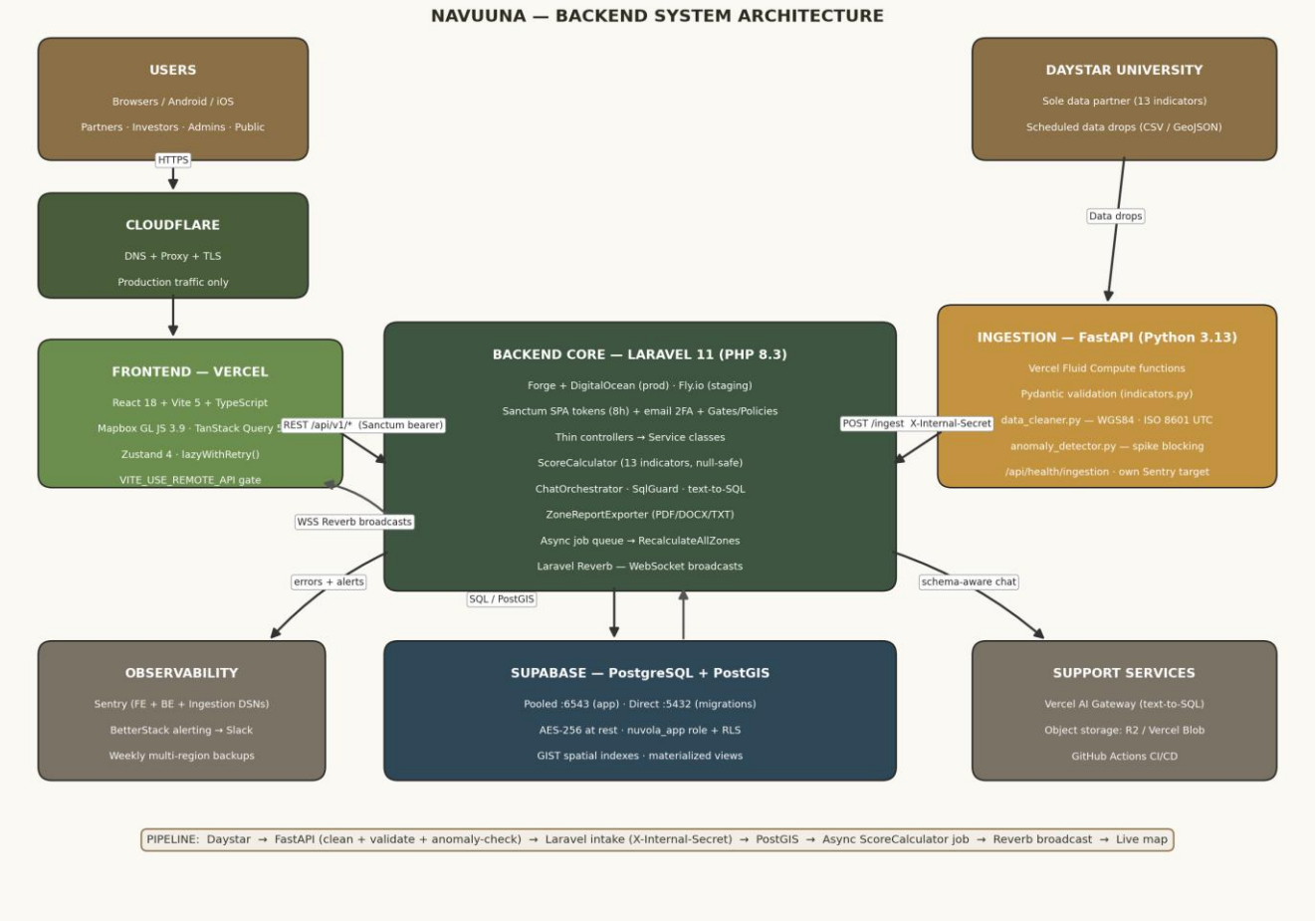

Figure 1 — Navuuna backend system architecture. Staging currently on Fly.io; production target remains Forge + DigitalOcean.

2.1 Layer Responsibilities

Layer	Technology	Responsibility
Edge	Cloudflare DNS + Proxy	Production traffic routing, TLS termination assist, DDoS shielding. Staging stays on the fly.dev subdomain until the production cutover.
Frontend	React 18 + Vite 5 on Vercel	Pure presentation. No business logic or DB calls in frontend files — hard rule. Talks to the backend only through /api/v1/*.
Core Platform	Laravel 11 (PHP 8.3+)	Auth (Sanctum, 8h TTL, email 2FA), Gates/Policies, thin controllers delegating to Service classes, async job queue, Reverb WebSocket broadcasts, PDF/DOCX/TXT export, text-to-SQL assistant.
Ingestion	FastAPI (Python 3.13)	Isolated intake engine: Pydantic validation, data_cleaner (WGS84, ISO 8601 UTC), anomaly_detector (spike blocking). Own Sentry target, own health endpoint.
Data	Supabase Postgres + PostGIS	AES-256 at rest, nuvola_app role with Row-Level Security, pooled :6543 for app traffic, direct :5432 for migrations, GIST spatial indexes, materialized views.
Realtime	Laravel Echo + Reverb	Score-change broadcasts pushed to live map sessions after async recalculation completes.

Layer	Technology	Responsibility
Observability	Sentry + BetterStack + Slack	Three independent DSNs (frontend, backend, ingestion). Threshold-triggered alerting into Slack. Weekly multi-region backups with monthly restore drills.

2.2 The Data Pipeline (End-to-End)

Every real data point on a user's screen travels this exact path. Each numbered stage is a hard boundary with its own validation and failure handling:

- 1. Daystar University delivers a data drop (CSV / GeoJSON) formatted against docs/data/daystar-indicator-spec.md.
- 2. FastAPI ingestion validates the payload with Pydantic, harmonizes coordinates to WGS84, forces ISO 8601 UTC timestamps, drops null components, and instantly rejects rows missing spatial anchors.
- 3. anomaly_detector.py scans for statistical spikes and blocks bad batches before they can touch the database.
- 4. The cleaned batch is POSTed to the Laravel intake endpoint, authenticated with the X-Internal-Secret header, with explicit retry limits and error fallbacks.
- 5. Laravel writes an append-only row to data_ingestion_logs (when it arrived, where it came from, field-verification status) and persists indicator values to PostGIS.
- 6. ScoreCalculator is dispatched as an asynchronous Laravel job (never called from a controller) and recomputes pillar and composite scores, excluding nulls.
- 7. On completion the job triggers a Reverb broadcast; open map sessions update instantly and the 'N of 13 indicators active' chip refreshes.

3. Non-Negotiable Backend Stack (Grant-Locked)

Layer	Technology / Restriction
Backend	Laravel 11 (PHP 8.3+)
Database	PostgreSQL + PostGIS (Supabase in prod, local Docker for tests)
Authentication	Sanctum SPA tokens (8h TTL) + email 2FA + roles + API keys
Backend Hosting	Target: Laravel Forge + DigitalOcean. Current staging: Fly.io (navuuna-atlas-staging.fly.dev)
DB Hosting	Supabase (Pooled :6543, direct :5432 for migrations)
Ingestion Engine	FastAPI (Python 3.13/3.14). Target: Vercel Functions with Fluid Compute
Realtime	Laravel Echo + Reverb
Error Tracking	sentry-laravel 4 (DSN-gated) + dedicated ingestion Sentry target
AI Gateway	Vercel AI Gateway (schema-aware text-to-SQL), USE_MOCK_CHAT gate while billing is pending
DNS / Proxy	Cloudflare

3.1 Stack Do-Nots (Hard Rules)

- No Next.js, Rails, or Django anywhere in the system.
- No MQTT — a longer-term architectural discussion, not a pilot deliverable.
- No Go migration or premature abstractions. No feature flags for hypothetical future needs.
- No database mocking in integration tests — Docker Postgres always.
- Never hardcode VITE_MAPBOX_ACCESS_TOKEN or any environment secret.
- No vector RAG until unstructured documents are ingested — structured RAG via text-to-SQL is the current pattern (app/Services/Chat/).
- Never add Co-Authored-By: Claude in git commit trailers.

4. Database Design

The data layer lives on Supabase PostgreSQL with the PostGIS extension enabled at container/instance start. All migrations must implement both up() and down(). Raw SQL is banned except isolated PostGIS spatial queries inside a dedicated service/repository method with an explanatory comment. The schema map below groups every table by domain; Phase E/F tables are included so the migration order is planned before build starts.

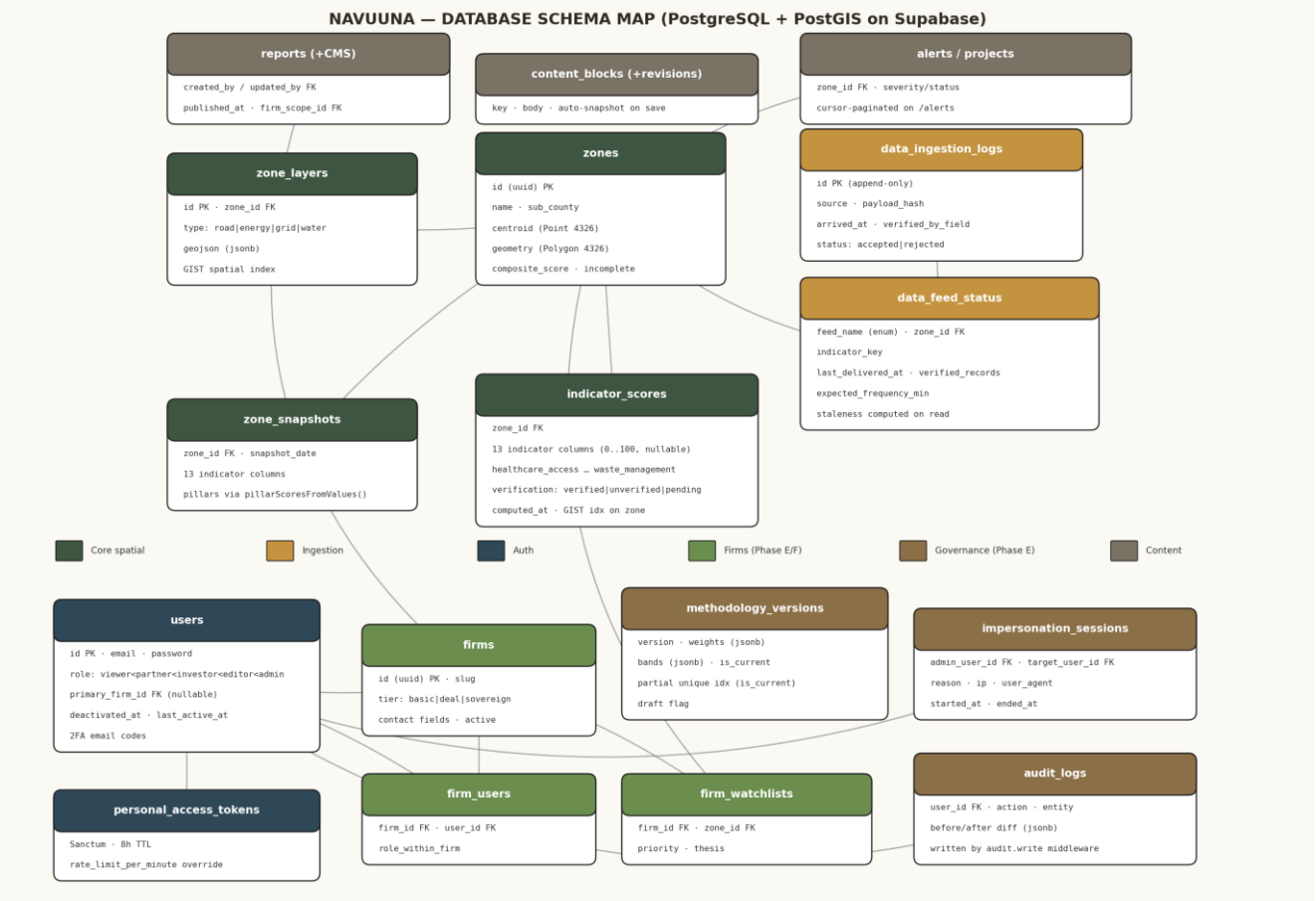

Figure 2 — Database schema map. Green = firm/investor tables (Phase E/F), brown = governance, amber = ingestion.

4.1 Core Spatial Tables

Table	Key Columns	Notes
zones	id (uuid) PK, name, sub_county, centroid Point(4326), geometry Polygon(4326), composite_score, incomplete flag	17 sub-counties seeded. GIST index on geometry. Composite score cached, recomputed by async job.
indicator_scores	zone_id FK + 13 nullable indicator columns (0..100) + verification enum + computed_at	Post-migration 2026_07_25_000001_swap_pillars_for_indicators. Nulls excluded from averages — never zero-biased.
zone_snapshots	zone_id FK, snapshot_date, 13 indicator columns	Powers /zones/{id}/history. Pillars derived at read time via ScoreCalculator::pillarScoresFromValues.
zone_layers	id PK, zone_id FK, type (road|energy|grid|water), geojson jsonb	Feeds atlas-map.sources.ts overlays. GIST spatial index required in Phase C.

4.2 Ingestion & Feed Health Tables

Table	Key Columns	Notes
data_ingestion_logs	id PK (append-only), source, payload_hash, arrived_at, verified_by_field, status	Append-only — no updates or deletes. The audit trail for 'when data arrived, where it came from, and whether field workers verified it'.
data_feed_status	feed_name enum, zone_id FK, indicator_key, last_delivered_at, verified_records, expected_frequency_min	Phase E migration. Staleness is computed on read by FeedStatusService — never stored. Powers the /admin/data feed × zone × indicator matrix.

4.3 Auth, Firms & Governance Tables (Phase E/F)

Table	Key Columns	Notes
users (extended)	+ primary_firm_id FK (nullable), deactivated_at, last_active_at	Migration extend_users_for_firms. Role hierarchy: viewer < partner < investor < editor < admin.
firms	id (uuid) PK, slug, tier (basic|deal|sovereign), contact fields, active	Investor firm entities. Seeded in dev with Acumen East Africa (deal), Andela Ventures (basic), GCF Nairobi Corridor (sovereign).
firm_users	firm_id FK, user_id FK, role_within_firm	Pivot table; a user's primary_firm_id is finalised by a follow-up FK migration.
firm_watchlists	firm_id FK, zone_id FK, priority, thesis	Investors see everything; watchlisted zones are highlighted, not filtered.
methodology_versions	version, weights jsonb, bands jsonb, is_current (partial unique idx), draft	Seed v1.0.0 is_current=true with equal 0.25 weights. ScoreCalculator reads is_current weights with a 60s cache instead of the hardcoded quartet.
impersonation_sessions	admin_user_id FK, target_user_id FK, reason, ip, user_agent, started_at, ended_at	Every impersonation is audited on both ends by ImpersonationService.
audit_logs	user_id FK, action, entity, before/after jsonb diff	Written by audit.write middleware on every admin action. CSV-exportable from /admin/audit.
content_blocks (+revisions)	key, body; revisions auto-snapshot on save	CMS backing for methodology copy and published content.
reports (extended)	+ created_by, updated_by, published_at, firm_scope_id	Migration extend_reports_for_cms. Enables firm-scoped report publishing.

4.4 Migration Order (Phase E backend)

Migrations must land in dependency order — the pivot and FK migrations rely on both parents existing:

- 1. extend_users_for_firms — add primary_firm_id, deactivated_at, last_active_at to users.
- 2. create_firms_table — uuid PK + slug + tier + contact fields + active.
- 3. create_firm_users_table — pivot with role_within_firm.
- 4. add_firm_fk_to_users — finalise users.primary_firm_id foreign key.
- 5. create_firm_watchlists_table — firm_id + zone_id + priority + thesis.
- 6. create_methodology_versions_table — weights jsonb + bands jsonb + is_current partial unique index; seed v1.0.0.
- 7. create_data_feed_status_table — feed_name enum + zone_id + indicator_key + delivery telemetry.
- 8. create_impersonation_sessions_table — full audit fields.

- 9. create_content_blocks_tables — content_blocks + content_block_revisions.
- 10. extend_reports_for_cms — CMS + firm scoping fields on reports.

5. Vitality Score Engine

The engine is App\Services\ScoreCalculator (the VitalityScoreService rewrite). It must never be called directly from a controller — it is dispatched strictly as an asynchronous Laravel job. All four pillars weight equally at 0.25; all legacy metrics (SPI, Optimal Density Ratio, Urban Friction Index, ESIA Transparency, Digital Sovereignty) are fully removed and must not be referenced.

5.1 The Four Pillars & Thirteen Indicators

Pillar (weight 0.25 each)	Indicators
1 — Social Wellbeing & Human Capital	healthcare_access · education_access · digital_connectivity
2 — Safety & Security	crime_rates · emergency_response_access · disaster_exposure
3 — Density & Scaling Dynamics	population_density · congestion · housing_pressure
4 — Infrastructure & Environmental Safeguards	road_quality · energy_reliability · food_risk · waste_management

Note the documented deviation: the grant note says twelve indicators, the implementation carries thirteen (Pillar 4 has four). The codebase is the source of truth going forward.

5.2 Calculation Rules

- Normalize each indicator on a strict 0 → 100 scale.
- Pillar Score = average of its normalized indicators, non-null only.
- Composite Score = average of the four pillar scores, skipping fully-null pillars.
- Missing data: exclude missing indicators from averages entirely — never treat them as zero. Mark the overall score incomplete if any indicator is absent.
- Visibility: ScoreCalculator::missingIndicators logs exactly which indicators are missing per locality and powers the '9 of 13 indicators active' UI badge.
- Phase E change: weights are read from methodology_versions WHERE is_current = true (60-second cache) so admins can retune and republish; publishing dispatches RecalculateAllZones.

6. Service Layer Map

Controllers stay thin; all business logic lives in Service classes. Permissions go through Laravel Gates and Policies only — no inline role checks.

Service / Middleware	Phase	Responsibility
ScoreCalculator	Shipped	13-indicator pillar + composite math, null exclusion, missingIndicators ledger. Needs async job dispatch wrapper (open item).
Chat services (AiGatewayClient, ChatOrchestrator, IntentRouter, SchemaCatalog v2, SqlGenerator, SqlGuard, SqlExecutor)	Shipped	Schema-aware text-to-SQL assistant behind the Vercel AI Gateway, mock-gated until billing lands.
Export\ZoneReportExporter	Shipped	PDF/DOCX/TXT zone reports; null pillars render as '—' not '0'. Extended in Phase F with the LP-style firm-portfolio brief.
Firms\FirmService	E	Firm CRUD + membership management.
Watchlist\WatchlistService	E	Firm watchlist writes (bulk PUT).
Methodology\MethodologyPublisher	E	Publish a methodology version + dispatch RecalculateAllZones.
Methodology\MethodologyPreview	E	Project scores under proposed weights before publishing.

Service / Middleware	Phase	Responsibility
Feeds\FeedStatusService	E	Feed health reads; staleness computed on read.
Impersonation\ImpersonationService	E	Mint target-user tokens; audit both ends of every session.
Content\ContentBlockService	E	CMS blocks with auto-snapshot revisions on save.
Middleware audit.write	E	Writes an audit_logs row per admin action.
Middleware firm.scope	E/F	Injects primary_firm_id + watchlist context; 403 for investors without a firm. Applied to all /investor routes.

7. API Surface

Every endpoint is prefixed /api/v1/*, authenticated with Sanctum bearer tokens, and returns the standard JSON shape — Success: { success: { status, data, message } }; Error: { error: { status, code, message } } conforming to RFC 7807 problem+json. The OpenAPI 3.1 spec at nuvola-atlas-backend/docs/api/openapi.yaml is authoritative. Pagination: cursor-based on /alerts and /zones/{id}/activity; page-based on /zones, /projects, /reports.

7.1 Core Platform Routes (shipped / Phase B)

Route	Method(s)	Status / Notes
/api/health	GET	Shipped — 200 with DB ping + cache checks, verified 2026-07-12.
/auth/* (sign-up, sign-in, email verify)	POST/GET	Shipped — email verification route registered; throttle keyed by email+IP.
/zones, /zones/{id}	GET	Shipped — ZoneResource exposes sub-metrics for freshness indicators.
/zones/{id}/history	GET	Shipped — averages 13 indicator columns, pillars via pillarScoresFromValues.
/ingest (internal)	POST	Phase B — accepts cleaned streams from FastAPI, X-Internal-Secret guarded.
/alerts, /projects, /reports	GET	Shipped — pagination per contract.

7.2 Admin Routes (Phase E — all under /api/v1/admin)

Route	Methods
/admin/firms · /admin/firms/{id}	GET, POST · GET, PATCH, DELETE
/admin/firms/{id}/users · /{userId}	POST · DELETE
/admin/firms/{id}/watchlist	PUT (bulk)
/admin/methodology · /{version}/preview · /{version}/publish	GET, POST · POST · POST
/admin/feeds · /admin/feeds/{feedName} · /admin/feeds/zones/{zoneId}	GET
/admin/impersonate/{userId} · /impersonate/end · /impersonations	POST · POST · GET
/admin/content/{key} · /{key}/revisions	GET, PUT · GET

7.3 Investor Routes (Phase F — firm.scope middleware on all)

Route	Purpose
/investor/me	Own profile + firm + tier.
/investor/watchlist	GET, POST/PATCH/DELETE per zone.
/investor/portfolio	Rollup: composite scores across watchlist + trend.
/investor/opportunities	Non-watchlisted zones ranked by tier-specific heuristic.
/investor/brief	LP-style PDF over the watchlist — extends ZoneReportExporter with the firm-portfolio format.

7.4 Ingestion Service Routes (FastAPI)

Route	Purpose
/api/health/ingestion	Shipped — service health check.
/ingest	Validated intake: Pydantic models → data_cleaner → anomaly_detector → forward to Laravel.

8. Security Architecture

Control	Implementation	Status
Transport & headers	HSTS enforced; CSP allows *.fly.dev, *.vercel.app, wss://*.fly.dev; Cloudflare in front of production.	Shipped (staging); prod DNS pending
Authentication	Sanctum SPA tokens with 8-hour TTL, custom email 2FA, admin 2FA reminder cron.	Shipped
Authorization	Role hierarchy viewer < partner < investor < editor < admin via Gates/Policies only. Supabase RLS on nuvola_app role (partner_overlays in place; investor firm scoping lands with Phase E).	Partial
Rate limiting	throttle:api 60/min per user or IP; per-token override via personal_access_tokens.rate_limit_per_minute; auth throttle 20 attempts / 10 min per email+IP; trust-proxy chain honours Fly/Cloudflare X-Forwarded-For.	Shipped
Internal channel	X-Internal-Secret header on the Python→PHP hop with explicit validation, retry limits, error fallbacks.	Phase B
Encryption at rest	Supabase managed disks AES-256 confirmed.	Shipped
Spend guards	Ingestion budget guards so a runaway background job cannot exhaust the API budget.	Phase C
Pen testing	Simulated attack with Strathmore Info Sec Club (or external specialists) across both codebases.	Phase C
Branch protection	main requires 1 manager approval + green CI, force-push disabled.	Open (Phase A remainder)
Secrets	Environment variables only, never in code. Rotation runbook shipped.	Shipped

9. Backend Build Phases & Task Checklist

The checklist below carries the tracker's exact state as of 2026-07-12 (HEAD 92626c3), filtered to backend and infrastructure work. Frontend-only items are excluded. Active phase: Phase B — blocked on Daystar delivery; Phases E and F design signed off, build pending.

Phase A Remainders — Backend Live (Jul 10 – Jul 24)

Why it's necessary: *Everything else in Phase A shipped on staging. These four items are what separates a Fly.io staging demo from a true production posture.*

- [ ] **Khillon:** Provision production Sentry targets and deploy active DSN keys to Forge and Vercel environments (package installed, DSN-gated; keys not issued).
- [ ] **Khillon:** Establish strict GitHub branch protection on main — 1 manager approval, green pipelines, no force-push.
- [ ] **Khillon:** Point live network traffic via Cloudflare DNS (production only; staging stays on fly.dev).
- [ ] **Khillon:** Execute the production Forge + DigitalOcean deploy (staging on Fly.io survives until then).
- [ ] **Devyan:** Validate and provision the parsing microservice's target architecture (Vercel Python Fluid Compute as blueprint).
- [ ] **Devyan:** Formulate a dedicated ingestion Sentry target and map independent keys.
- [ ] **Devyan:** Build the Docker Compose orchestrator tying local engines together (FastAPI + Laravel + PostGIS + Reverb + Nginx).

Phase B — Real Data Ingestion (Aug 01 – Aug 21) — ACTIVE, blocked on Daystar

Why it's necessary: *Disconnects the platform from fixtures.ts entirely. The ingestion scaffold is complete on our side; external data delivery has not started. This is the single largest schedule risk in the project — no fallback ingestion path exists.*

- [ ] **Devyan & Khillon:** Formalize internal transmission signatures between Python and PHP: X-Internal-Secret tokens, validation parameters, error fallbacks, retry limits.
- [x] **Devyan:** Draft the Daystar indicator spec for all 13 parameters (docs/data/daystar-indicator-spec.md).
- [ ] **Devyan:** Supply spec copies to Joy to drive coordination with Daystar administrators.
- [ ] **Devyan:** Evaluate Daystar capabilities vs. platform requirements; register coverage limits and integration holes.
- [ ] **Devyan:** Feed deficit lists to Khillon to calibrate partial-scoring around field realities.
- [ ] **Devyan:** Initialize granular intake jobs as streams clear validation — do not wait for full rollouts.
- [ ] **Devyan:** Add ruff + mypy config to the ingestion pipeline (pytest already present).
- [x] **Devyan:** Health endpoint, Pydantic models, data_cleaner.py, anomaly_detector.py (statistical; TF/PyTorch deferred) — all shipped in the scaffold.
- [ ] **Devyan:** Enforce automated cron scheduling for intake parsing scripts.
- [ ] **Devyan:** Run the full end-to-end telemetry sweep: Daystar → FastAPI → Laravel → PostGIS → WebSocket → live screens.
- [ ] **Khillon:** Code intake routing hooks accepting cleaned streams, guarded by strict X-Internal-Secret checks.
- [ ] **Khillon:** Structure the append-only data_ingestion_logs table (design captured with Phase E's data_feed_status).
- [x] **Khillon:** Zone detail + history endpoints exposing sub-metrics and 13-indicator-derived pillars.
- [ ] **Khillon:** Wrap ScoreCalculator::recalculate in an async job dispatch + Reverb broadcast on status shifts (currently synchronous — violates the coding rule).

- [x] **Khillon:** Production-ready GeoJSON features for Roads, Energy, Density (zone_layers + atlas-map.sources.ts).

Phase C — Hardening & Operations (Aug 22 – Sep 05)

Why it's necessary: *Optimizes database scaling and locks down operational posture before real load arrives.*

- [ ] **Khillon:** Definitive design sweep over active PostGIS tables before accepting production feeds.
- [ ] **Khillon:** GIST spatial indexes across all core spatial columns.
- [ ] **Khillon:** Pre-aggregated materialized views for county-wide status, refreshed by overnight routines.
- [ ] **Khillon:** Object storage decision: Cloudflare R2 vs Vercel Blob for large document exports.
- [ ] **Khillon:** Pruning routines: scrub raw storage text files after 30 days; lock analytical scores indefinitely.
- [x] **Khillon:** Endpoint throughput constraints (throttle:api + per-token overrides + auth throttle) — shipped early.
- [ ] **Khillon:** Penetration evaluation with Strathmore Info Sec Club or external specialists.
- [ ] **Khillon:** BetterStack centralized error tracking with threshold-triggered notifications.
- [ ] **Khillon:** Weekly multi-region backups + monthly restore drills.
- [ ] **Devyan:** Ingestion spend guards so a runaway job cannot blow the API budget.

Note: PgBouncer deployment was judged superfluous — Supabase's pooler on :6543 already covers connection scaling.

Phase D — Validation & Launch Readiness (Sep 06 – Sep 18)

Why it's necessary: *The six pilot-ready criteria must all evaluate true against live production feeds before stakeholder demos.*

- [ ] Authentication: partners log in cleanly with roles validated (verified on staging 2026-07-12; needs production verification).
- [ ] Live Spatial Data: map renders live Daystar-derived points, not fixtures.ts.
- [ ] Scoring Authenticity: real index measurements with partial-data boundaries clearly communicated.
- [ ] Transparency: methodology modules explain computation factors, dataset origins, sync times.
- [x] Portability: client-side PDF generation from scorecard states (ZoneReportExporter shipped).
- [ ] Predictability: ingestion schedules match formalized partnership agreements.
- [ ] **Khillon:** Final review: field metrics, schema docs, 100% green suites, final merge confirmations.
- [ ] **Devyan:** Ingestion validation + pipeline architecture docs + health diagnostics to the board.

Phase E — Admin Suite Backend (design signed off 2026-07-12)

Why it's necessary: *Gives internal staff the deep-controls console: users, firms, Daystar delivery health, scoring-engine tuning, audit trail, content management.*

Migrations (in the dependency order of §4.4):

- [ ] **Khillon:** All ten Phase E migrations: extend_users_for_firms → firms → firm_users → firm FK → firm_watchlists → methodology_versions (seed v1.0.0) → data_feed_status → impersonation_sessions → content_blocks(+revisions) → extend_reports_for_cms.

Services & middleware:

- [ ] **Khillon:** FirmService · WatchlistService · MethodologyPublisher (+RecalculateAllZones dispatch) · MethodologyPreview · FeedStatusService · ImpersonationService · ContentBlockService.
- [ ] **Khillon:** audit.write middleware (audit row per admin action) and firm.scope middleware (403 for investors without a firm).

- [ ] **Khillon:** Modify ScoreCalculator to read weights from methodology_versions WHERE is_current = true with a 60s cache.

Routes:

- [ ] **Khillon:** Full /admin route family per §7.2 — firms, methodology, feeds, impersonation, content.

Seed data:

- [ ] **Khillon:** FirmSeeder (Acumen/Andela/GCF), FirmUserSeeder (investor+{firm}@navuuna.dev), FirmWatchlistSeeder, FeedStatusSeeder with realistic mixed staleness.

Phase F — Investor Suite Backend (design signed off 2026-07-12)

Why it's necessary: *A purpose-built read-only surface framing Navuuna's data for capital allocation. Watchlist scoping: investors see everything; watchlisted zones are highlighted.*

- [ ] **Khillon:** /investor/me — own profile + firm + tier.
- [ ] **Khillon:** /investor/watchlist — GET, POST/PATCH/DELETE per zone.
- [ ] **Khillon:** /investor/portfolio — composite score rollup across watchlist + trend.
- [ ] **Khillon:** /investor/opportunities — non-watchlisted zones ranked by tier-specific heuristic.
- [ ] **Khillon:** /investor/brief — LP-style PDF extending ZoneReportExporter with the firm-portfolio format.
- [ ] **Khillon:** Apply firm.scope middleware to every investor route.

10. Test Baseline & Definition of Done

Run immediately after every meaningful feature slice. Nothing is 'done' until all five checks are green. Backend tests always run against Docker Postgres — never mocked databases.

```
1. cd nuvola-atlas-frontend && npx tsc --noEmit
2. cd nuvola-atlas-frontend && npx vite build
3. cd nuvola-atlas-frontend && npx vitest run
4. cd nuvola-atlas-backend && php artisan route:list --path=api
5. cd nuvola-atlas-backend && php vendor/phpunit/phpunit/phpunit --no-coverage
   # Requires: docker compose up -d postgres  (from the backend directory)
```

Last full green baseline: HEAD c32002e (2026-06-27) — tsc clean · vite build ~15s · vitest 15/15 · route:list 32 · phpunit 91/91 (367 assertions) ~16s. The backend suite was fully rebased for the 13-indicator schema (Support/IndicatorSeeding helper + Feature test sweep) and requires a re-run against Docker Postgres to confirm green — until then, the CI backend job stays continue-on-error (informational) while the frontend job remains enforcing.

Coding rules recap (non-negotiable)

- Secrets from environment variables only. No inline role checks — Gates and Policies.
- Thin controllers; business logic in Service classes. Reversible migrations (up + down).
- No raw SQL except isolated, commented PostGIS spatial queries in a dedicated service/repository method.
- Consistent JSON envelope on every endpoint (RFC 7807 for errors).
- ScoreCalculator dispatched as an async job only. No commented-out code. PSR-12 (PHP) / PEP 8 (Python).
- Conventional Commits, per-slice, push only when green. No PR merges to main without passing CI.
- Plan Mode for anything ≥ 3 steps or with architectural tradeoffs; stop and ask on ambiguity.

11. Risk Register

Risk	Severity	Mitigation
Daystar delivery slips — all 13 indicators sourced exclusively through this partnership; no fallback path exists.	Critical	Partial-score logic already ships (nulls excluded, 'N of 13' chip). Initialize granular intake per-stream as feeds clear validation. Escalate coordination through Joy. Phase B cannot fully close without confirmed delivery across all four pillars.
Synchronous ScoreCalculator under load.	High	Wrap in async job dispatch (open Phase B item) before real feeds land; Reverb broadcast on completion.
Backend phpunit not re-verified green post-schema-swap.	High	Re-run the 5-check baseline against Docker Postgres; lift continue-on-error on the CI backend job once green.
Supabase migrations table drift (repaired 2026-07-12 via fix-indicators.sql).	Medium	Idempotent SQL captured; enforce artisan migrate as the only DDL path going forward.
Runaway ingestion job exhausts API budget.	Medium	Phase C spend guards on the ingestion service.
main unprotected until branch rules land.	Medium	Phase A remainder — 1 approval + green CI + no force-push.
Production cutover (Fly → Forge + DO) introduces config drift.	Medium	Dockerfile maintained for Fly/Forge parity; .env.production.example as the contract; re-verify the six pilot criteria on production.

12. Completion & Sign-off

- [ ] Phase A Complete: infrastructure active on production (Forge + DO deploy still open; staging complete).
- [ ] Phase B Complete: fixtures.ts fully decoupled from production routing (blocked on Daystar delivery).
- [ ] Phase C Complete: security, scaling, and performance configurations verified.
- [ ] Phase D Complete: all six validation checkpoints verified on real-world data.
- [ ] Phase E Complete: admin migrations, services, routes shipped and green.
- [ ] Phase F Complete: investor backend shipped and green.
- [ ] Integrated test suites 100% green across all environments; topography docs match the live repo.
- [ ] Unified executive board sign-off executed per team governance frameworks.

End of Backend Build Plan — Navuuna Technical Series.

NAVUUNA

Backend Build Plan & System Architecture — Version 1.1

Making Africa Investment Green

AI, ML, RAG, Agents & Automation Expansion

Version 1.1 · July 13, 2026 · Supersedes v1.0 (July 12, 2026, HEAD 92626c3) · Adds Phases G–J and Appendix A (Claude Code Workstream Prompt)

What changed in v1.1	Where
Machine-learning layer: Vitality Forecaster, anomaly_detector v2, Opportunity Ranker, Data Quality Classifier — served from the existing FastAPI service	§13 · Phase G
Retrieval-Augmented Generation on Supabase pgvector, hybrid with the shipped text-to-SQL path, gated behind the existing 'no vector RAG until unstructured docs exist' rule	§14 · Phase H
Agentic chatbot: ChatOrchestrator evolves into a bounded agent loop with a role-scoped tool registry; four scheduled background agents	§15 · Phase I
Automation layer: decision rule for Laravel Scheduler/Horizon vs self-hosted n8n, plus six priority workflows including the Daystar intake automation	§16 · Phase J
New database tables, API routes, security controls, phase checklists and risk-register entries for the AI workstream	§17–§19
Ready-to-paste Claude Code prompt covering the entire repo context and this expansion	Appendix A

13. Purpose of this Expansion

Sections 1–12 of the Backend Build Plan (v1.0) remain in force unchanged: the Laravel 11 core platform, the FastAPI ingestion engine, the Supabase PostgreSQL/PostGIS data layer, the Vitality Score engine (4 pillars × 13 indicators, equal 0.25 weights, nulls excluded), the Phase A–F checklists, and every grant-locked stack rule. This expansion answers one question: **how Navuuna becomes an intelligent platform rather than a reporting platform** — predictive scores instead of only current scores, grounded answers instead of only SQL lookups, agents that act instead of a chatbot that replies, and automations that remove the manual glue currently holding Phase B together.

Authority chain is unchanged: tracker > this plan > codebase deviations logged. Nothing in Phases G–J may begin before its stated entry condition, and nothing here overrides the Stack Do-Nots in §3.1 — in particular, the 'no vector RAG until unstructured documents are ingested' rule is **formalised** here as the Phase H entry gate, not lifted.

13.1 Design Principles for the AI Workstream

- **Extend, never rebuild.** The shipped text-to-SQL stack (AiGatewayClient, ChatOrchestrator, IntentRouter, SchemaCatalog v2, SqlGenerator/SqlGuard/SqlExecutor) remains the default path for structured questions. Every new capability wraps around it.
- **Python runs models, PHP runs the product.** All ML training and inference lives in the FastAPI service; Laravel consumes predictions through thin, X-Internal-Secret-guarded Service classes. No PHP ML libraries.
- **Everything auditable.** Every model version, prediction, agent step, and automation run is a database row. This mirrors the append-only data_ingestion_logs philosophy and feeds the /admin audit surface.
- **Budget-guarded by construction.** The Phase C spend-guard pattern extends to every LLM and ML call path. USE_MOCK_CHAT-style env gates for anything with unconfirmed billing.
- **Permissions in code, not prompts.** Agent tools are wrappers over existing Service classes so Gates/Policies and firm.scope apply automatically. A prompt never enforces security; SqlGuard, allow-lists, and RLS do.

14. Phase G — Machine Learning Layer

Entry condition: Phase B streaming real Daystar data for at least one pillar, and zone_snapshots accumulating history (forecasting is meaningless on fixtures). **Owner:** Devyan (models, FastAPI serving) + Khillon (Laravel consumption, tables, admin surface).

14.1 The Four Models

Model	Task & Approach	Consumed by
Vitality Forecaster	Time-series forecasting over zone_snapshots (13 indicator columns per zone per snapshot_date). v1: gradient-boosted trees (LightGBM) with lag/rolling features, or classical SARIMA/ETS per indicator-zone pair where history is thin. Deep learning deferred until data volume justifies it — consistent with the 'no premature abstractions' rule. Output: 30/90-day projected indicator, pillar and composite scores with confidence intervals; projections through the shipped pillarScoresFromValues math so forecast and live scores can never disagree on methodology.	Map 'projected score' layer; /investor/portfolio trend; Investor Digest Agent
anomaly_detector v2	Upgrade of the shipped statistical spike-blocker: IsolationForest + seasonal-decomposition residual scoring, trained on accepted historical batches (data_ingestion_logs × indicator_scores). The statistical detector remains as a hard fallback and both must agree to auto-reject; disagreement flags for human review instead. TF/PyTorch remain deferred exactly as noted in the Phase B scaffold.	Ingestion pipeline stage 3 (unchanged position)
Opportunity Ranker	Ranking model behind /investor/opportunities. v1 ships as a transparent tier-aware weighted heuristic (documented weights per basic|deal|sovereign); v2 moves to learning-to-rank once click/watchlist interaction data exists. The v1→v2 seam is one Service method — no premature abstraction.	/investor/opportunities (Phase F route)
Data Quality Classifier	Grades every Daystar batch on completeness, drift vs. historical distribution, and plausibility; writes quality_grade + reasons jsonb onto data_ingestion_logs. Powers the /admin/data matrix and the Data Quality Agent.	Admin console; Data Quality Agent

14.2 Serving, Registry & Retraining

- New FastAPI routes — **/ml/forecast**, **/ml/rank**, **/ml/quality**, **/ml/retrain** — internal only, X-Internal-Secret guarded, own Sentry breadcrumbs on the existing ingestion DSN.
- Laravel consumes via thin Service classes: Ml\ForecastService, Ml\RankService, Ml\QualityService. Controllers never call FastAPI directly.
- **ml_models registry table** (§17): every trained model is a versioned row with metrics jsonb and an artifact URI in object storage (R2 or Vercel Blob per the Phase C decision — one bucket, models/ prefix). is_current enforced with a partial unique index, mirroring methodology_versions.
- Every inference writes an **ml_predictions** row (model FK, zone FK, horizon, payload) so any number shown to an investor is traceable to the exact model version that produced it.
- **Champion/challenger retraining:** scheduled retrain job evaluates the new candidate on a held-out window; it only becomes is_current after beating the incumbent AND receiving human approval (Slack approval step — automation workflow #6, §16.3). No silent model swaps.
- Determinism in tests: every ML endpoint ships a frozen fixture model so the phpunit/pytest suites stay reproducible against Docker Postgres.

15. Phase H — Retrieval-Augmented Generation

Entry gate (formalising the §3.1 rule): Phase H may not begin until unstructured documents actually exist in the system — content_blocks/CMS methodology copy (lands with Phase E), published reports, the Daystar indicator spec, and partner documents. Until that gate opens, structured RAG via text-to-SQL remains the only retrieval pattern.

15.1 Architecture

- **pgvector on Supabase.** The vector store is an extension on the existing grant-locked database — no new vendor, no separate vector DB. Enabled the same way PostGIS is: at instance start.
- **document_chunks table** (§17): source_type + source_id polymorphic reference, chunk_text, embedding vector, metadata jsonb (zone ids, firm scope, doc version), embedded_at. HNSW index for ANN search.
- **Indexing pipeline as a Laravel job** — Rag\DocumentIndexer, dispatched on content_block save, report publish, and methodology publish (piggybacking the same events that already dispatch RecalculateAllZones). Embeddings generated through the existing Vercel AI Gateway — same gateway, same env-only keys, same USE_MOCK_CHAT-style gate until billing is confirmed.
- **Hybrid retrieval routed by IntentRouter:** numeric/structured question → text-to-SQL (unchanged); narrative/'why'/methodology question → vector retrieval → grounded answer; mixed question → both branches, merged by ChatOrchestrator into one response.
- **Citations mandatory.** Every RAG answer cites its source chunks (doc title + section). If top-k similarity falls below threshold, the assistant says it doesn't have a grounded answer rather than improvising — the same honesty rule as the 'N of 13 indicators' chip.
- **Scoped retrieval.** Firm-scoped reports are only retrievable by that firm's investors. Enforcement is the firm.scope middleware + Supabase RLS on document_chunks — never a prompt instruction. Public users retrieve public chunks only.

15.2 What RAG Deliberately Does Not Do

- No general web knowledge — the corpus is Navuuna's own documents. Out-of-corpus questions route to text-to-SQL or a polite refusal.
- No fine-tuning in Phase H. Retrieval over a curated corpus with a strong base model is cheaper, auditable, and reversible.
- No cross-firm leakage paths: chunk metadata carries firm_scope_id and RLS filters at the database layer, so even a compromised prompt cannot widen retrieval.

16. Phase I — Agentic Chatbot & Background Agents

Entry condition: Phase E Gates/Policies and audit.write middleware shipped (the tool registry depends on them), and AI Gateway billing confirmed (the mock gate lifts). Phase H is a soft dependency — the agent ships without the search_docs tool and gains it when RAG lands.

16.1 From Pipeline to Agent Loop

Today ChatOrchestrator executes one fixed path per turn: intent → SQL → answer. Phase I evolves it into a **bounded agent loop** — plan → select tool → act → observe → repeat — with three hard limits: a maximum step count, a wall-clock timeout, and a per-conversation token + tool-call budget. When any limit is hit, the loop degrades gracefully to the best plain answer available. The loop is a Service class (Chat\AgentLoop) so controllers stay thin per the coding rules.

16.2 Tool Registry (role-scoped, allow-listed)

Tool	Wraps (existing service)	Roles	Write?
query_data	SqlGenerator → SqlGuard → SqlExecutor	all authed	No
search_docs (Phase H)	Rag retrieval, RLS-scoped	all authed	No
get_zone / compare_zones	ZoneResource + history endpoints	all authed	No
get_feed_status	FeedStatusService	editor, admin	No
forecast_zone (Phase G)	Ml\ForecastService	investor+, admin	No
export_report	ZoneReportExporter	partner+	Yes — confirm turn
manage_watchlist	WatchlistService via firm.scope	investor w/ firm	Yes — confirm turn

- Each tool is a thin wrapper over a Service class, so **Gates/Policies apply automatically** — a viewer's agent literally cannot see investor tools; there is no tool to jailbreak into.
- **Write-capable tools require an explicit confirmation turn:** the agent proposes the action ('Add Westlands to your watchlist with priority 2?'), the user confirms, then it executes. No silent writes, ever.
- **Every step is logged** to agent_runs / agent_steps (tool, args, result hash, tokens, latency) — the chat equivalent of audit_logs, CSV-exportable from the admin console.
- **Prompt-injection posture:** SQL results and retrieved chunks are wrapped as delimited data, never interpreted as instructions; the enforcement layer is SqlGuard + the allow-list + RLS, not prompt wording. A red-team pass on the agent joins the Phase C Strathmore Info Sec Club engagement scope.

16.3 Background Agents (scheduled, not chat-driven)

Agent	Trigger	What it does
Feed Watchdog	Laravel Scheduler, hourly	Reads FeedStatusService, detects staleness patterns across the feed × zone × indicator matrix, drafts a diagnosis ('education_access stale in 4 of 17 zones since Tuesday — pattern suggests a partial Daystar drop') and hands it to the escalation workflow (§17.3 #2) instead of a bare threshold ping.
Data Quality	On flagged batch (Phase G classifier)	Summarises per-batch what failed and why in plain language for the /admin/data console; proposes accept/reject with reasons. Human decides.
Investor Digest	Weekly, per firm	Watchlist score movements, newly ranked opportunities, one-paragraph narrative; rendered through ZoneReportExporter (extends the Phase F LP-style brief). Delivery via automation workflow #3.

Agent	Trigger	What it does
Ops Reporter	Monday 07:00 EAT	Internal digest: ingestion volume, Sentry error classes, pipeline health, phase-checklist deltas — posted to Slack.

Each background agent is a Laravel job chain that may call the LLM through AiGatewayClient; agents share the same budget guards, run ledger, and mock gates as the chat agent. An agent that only formats data (Ops Reporter) should be implemented without an LLM call at all where a template suffices — capability is not a reason to spend tokens.

17. Phase J — Automation Layer (n8n + Laravel-native)

17.1 The Decision Rule

One rule decides where any automation lives. **Inside the trust boundary → Laravel Scheduler/Horizon.** Anything touching application data, requiring app permissions, or mutating state runs as a first-party job: score recalculation, materialized-view refresh, pruning routines, snapshot generation, agent schedules.
Cross-service glue → n8n. Anything stitching external systems together — email, Slack, shared drives, Sentry webhooks, human-in-the-loop approvals — runs as an n8n workflow. This keeps business logic in tested PHP and keeps brittle third-party plumbing in a tool built for it.

n8n deployment & containment: self-hosted (Docker) on the same DigitalOcean box or a small Fly.io app, behind Cloudflare Access — never publicly exposed. n8n talks to Navuuna **only through the public /api/v1/* surface** with a dedicated 'automation' Sanctum token (per-token rate-limit override via personal_access_tokens), and to FastAPI only through its public health/ingest routes. Direct database access from n8n is banned — same class of rule as 'no business logic in the frontend'. Every workflow execution reports into automation_runs (§18) via a completion webhook, so ops visibility stays inside the admin console.

17.2 Why n8n (and the alternative considered)

- Self-hostable (data sovereignty for a civic-data platform; no per-task SaaS pricing like Zapier/Make), source-available, huge node library (Gmail/Drive/Slack/webhooks), and versionable — workflows export as JSON into the repo under infra/n8n/.
- Alternative considered: pure Laravel Scheduler for everything. Rejected for cross-service glue because email-attachment watching, Slack approval buttons, and webhook fan-in are exactly the code nobody should hand-write and maintain; kept for everything inside the trust boundary.
- Kill-switch: every workflow idempotent (payload_hash dedupe in automation_runs) and individually disableable; n8n outage degrades to the current manual process, never to data loss.

17.3 Priority Workflows

#	Workflow	Flow	Why it matters
1	Daystar drop intake	Watch drop channel (email attachment / shared-drive webhook) → validate filename against daystar-indicator-spec.md → POST to FastAPI /ingest → post validation summary + 'N of 13 indicators active' delta to Slack #data-feeds	Removes the manual hand-off on the critical path of Phase B — the single largest schedule risk in the project.
2	Feed staleness escalation	Feed Watchdog Agent output → warn Slack at T+expected_frequency → email Joy at 2× → create tracker task at 3×	Turns feed health from a dashboard someone must remember to check into an escalation ladder.
3	Investor digest dispatch	Laravel renders (Investor Digest Agent) → n8n delivers per firm, handles bounces/unsubscribes → delivery receipts to automation_runs	Keeps deliverability plumbing out of PHP.
4	Backup & restore-drill verification	Monthly trigger → restore drill on scratch instance → checksum comparison → pass/fail evidence to Slack	Converts the Phase C 'monthly restore drills' checkbox into a machine-verified ritual.
5	Sentry triage	Sentry webhook (3 DSNs) → dedupe → severity rubric → tracker item for new error classes only	Threshold alerting (BetterStack) stays; this removes duplicate-noise triage.
6	Model retraining approval	Cron → FastAPI /ml/retrain → champion/challenger result → Slack approval buttons → on approve, flip is_current in ml_models	Human-in-the-loop gate for Phase G — no silent model swaps.

18. Schema, API & Security Additions

18.1 New Tables (migration dependency order)

Table	Key Columns	Notes
conversations	id uuid PK, user_id FK, title, created_at	Chat persistence — the agent finally gets memory across turns server-side.
conversation_messages	conversation_id FK, role enum, content, tool_calls jsonb, tokens	Token column feeds the per-conversation budget guard.
ml_models	name, version, task enum(forecast|anomaly|rank|quality), metrics jsonb, artifact_uri, is_current (partial unique idx), trained_at	Mirrors methodology_versions pattern; artifacts in R2/Vercel Blob.
ml_predictions	model_id FK, zone_id FK, horizon, payload jsonb, created_at	Every surfaced number traceable to its model version.
agent_runs	id uuid, agent enum, trigger enum(chat|schedule), user_id nullable FK, status, token_cost, started_at, ended_at	The audit_logs of the agent layer.
agent_steps	agent_run_id FK, seq, tool, args jsonb, result_hash, latency_ms	Step-level ledger; CSV-exportable from /admin.
automation_runs	workflow_key, external_id (n8n execution id), status, payload_hash, ran_at	payload_hash gives idempotency/dedupe for workflows.
document_chunks (Phase H)	source_type, source_id, chunk_text, embedding vector, metadata jsonb (incl. firm_scope_id), embedded_at	pgvector + HNSW index; RLS-scoped.

All migrations reversible (up + down), applied via artisan migrate only (per the 2026-07-12 drift repair), RLS reviewed per table before merge.

18.2 New API Surface

Route	Method(s)	Notes
/chat/conversations · /{id}/messages	GET, POST	Agentic chat; SSE/Reverb streaming of steps + tokens.
/zones/{id}/forecast	GET	Phase G projections + confidence intervals; role-gated.
/admin/agents · /agents/{runId}	GET	Agent run ledger, filter by agent/status; CSV export.
/admin/models · /models/{id}/activate	GET · POST	Registry view; activate = human side of workflow #6.
/admin/automations	GET	automation_runs surface for the admin console.
/webhooks/n8n/{workflowKey}	POST	Signed completion callbacks (HMAC, shared secret env).
FastAPI: /ml/forecast · /ml/rank · /ml/quality · /ml/retrain	POST	Internal only, X-Internal-Secret; never exposed via Cloudflare.

18.3 Security Additions

Control	Implementation
LLM spend guards	Per-conversation token budget + daily gateway ceiling; extends the Phase C ingestion spend-guard pattern to the Vercel AI Gateway. Hard stop, alert to Slack.
Tool allow-listing	Tools resolved per role via Gates before the model ever sees the tool list; write tools additionally require a user confirmation turn.

Control	Implementation
Prompt-injection defense	Retrieved chunks / SQL results delimited as data; SqlGuard remains read-only + table-allow-listed; RLS bounds retrieval. Agent red-teaming added to the Phase C pen-test scope.
n8n containment	Cloudflare Access in front; API-token-only access to Navuuna; no DB credentials in n8n; workflow JSON reviewed in PRs like code.
Model provenance	ml_models + ml_predictions give full lineage; champion/challenger + human approval before any is_current flip.
Webhook integrity	HMAC-signed n8n callbacks; replay protection via payload_hash uniqueness.

19. Updated Phases, Checklists & Risks

19.1 Phase Entry Conditions & Sequencing

Phase	Entry condition	Indicative window
G — ML Layer	Phase B streaming ≥ 1 pillar of real data; zone_snapshots accumulating	Oct 2026 (post-Phase D)
H — RAG	Unstructured corpus exists (Phase E CMS + published reports + specs)	Nov 2026
I — Agentic	Phase E Gates/audit shipped; AI Gateway billing confirmed (mock gate lifts)	Nov–Dec 2026
J — Automation	Workflow #1 may start immediately (unblocks Phase B); rest roll out with their dependencies	Jul 2026 → rolling

Note the deliberate exception: automation workflow #1 (Daystar drop intake) is pulled forward of everything else because it attacks the project's critical-path risk today.

19.2 Backend Task Checklist Additions

Phase G — ML Layer

- [ ] Devyan: Feature pipeline over zone_snapshots (lags, rolling stats); frozen fixture dataset for tests.
- [ ] Devyan: Vitality Forecaster v1 (LightGBM / classical per thin-history pairs) + /ml/forecast.
- [ ] Devyan: anomaly_detector v2 (IsolationForest + residuals); statistical detector retained as fallback; agree-to-reject logic.
- [ ] Devyan: Data Quality Classifier + quality_grade writeback to data_ingestion_logs.
- [ ] Devyan: /ml/retrain with champion/challenger evaluation + held-out window.
- [ ] Khillon: ml_models + ml_predictions migrations; Ml* Service classes; /zones/{id}/forecast; /admin/models routes.
- [ ] Khillon: Opportunity Ranker v1 heuristic behind /investor/opportunities with documented tier weights.

Phase H — RAG

- [ ] Khillon: Enable pgvector; document_chunks migration + HNSW index + RLS (firm_scope_id).
- [ ] Khillon: Rag\DocumentIndexer job wired to content_block save / report publish / methodology publish events.
- [ ] Khillon: IntentRouter v2 (structured vs narrative vs mixed); citation-mandatory answer assembly in ChatOrchestrator.
- [ ] Khillon: Low-confidence refusal path + retrieval-scope Policy tests (cross-firm leakage tests are blocking).

Phase I — Agentic

- [ ] Khillon: conversations + conversation_messages + agent_runs + agent_steps migrations.
- [ ] Khillon: Chat\AgentLoop (max steps, timeout, token budget, graceful degrade) + tool registry with per-role Gate resolution.
- [ ] Khillon: Confirmation-turn protocol for write tools; step streaming over Reverb.
- [ ] Khillon: Background agents: Feed Watchdog, Data Quality, Investor Digest, Ops Reporter (template-first, LLM only where it adds value).
- [ ] Both: Agent red-team scenarios added to the Phase C pen-test scope with Strathmore Info Sec Club.

Phase J — Automation

- [ ] Devyan: n8n self-hosted deploy (Docker, Cloudflare Access); infra/n8n/ workflow JSON in repo; automation Sanctum token issued.

- [ ] Devyan: Workflow #1 Daystar drop intake — PULLED FORWARD, starts now.
- [ ] Khillon: automation_runs migration + /webhooks/n8n/{workflowKey} (HMAC) + /admin/automations.
- [ ] Both: Workflows #2–#6 rolled out with their phase dependencies; each idempotent and individually disableable.

19.3 Risk Register Additions

Risk	Severity	Mitigation
LLM/ML spend runs away once mock gates lift	High	Per-conversation + daily budget guards, hard stop + Slack alert; template-first rule for background agents; USE_MOCK_CHAT-style gates stay until billing confirmed.
Prompt injection widens agent capability	High	Enforcement in code (SqlGuard, allow-lists, RLS, confirmation turns), never in prompts; red-team pass in Phase C pen test.
Forecasts on thin history mislead investors	High	Confidence intervals mandatory in UI; forecasts suppressed below a minimum-history threshold; 'projected' visually distinct from live scores; model version disclosed.
n8n becomes an unaudited side-channel	Medium	API-token-only access, no DB credentials, HMAC callbacks, automation_runs ledger, workflow JSON in PR review.
RAG leaks firm-scoped content	Medium	RLS on document_chunks + blocking cross-firm leakage tests before Phase H merge.
Model drift after Daystar methodology changes	Medium	Data Quality Classifier drift signal triggers retrain workflow; champion/challenger prevents regression promotion.
Agent layer built before Phase B–E foundations	Medium	Hard entry conditions in §19.1; tracker discipline — phases G–J cannot open early.

19.4 Updated Completion & Sign-off

- [ ] Phase G Complete: four models registered, served, consumed; predictions ledgered; retraining gated.
- [ ] Phase H Complete: hybrid retrieval live with citations; scope tests green; entry gate honoured.
- [ ] Phase I Complete: agent loop + tool registry + four background agents shipped; agent_runs auditable from /admin.
- [ ] Phase J Complete: workflows #1–#6 live, idempotent, ledgered; workflow #1 verified against a real Daystar drop.
- [ ] AI workstream red-team findings closed or accepted with sign-off.

Appendix A — Claude Code Workstream Prompt

Paste the prompt below into Claude Code at the start of any AI-workstream session (or commit it to the repo as docs/ai/CLAUDE_AI_WORKSTREAM.md and reference it from CLAUDE.md). It is also shipped alongside this PDF as CLAUDE_CODE_PROMPT.md for direct copy-paste.

```
# CLAUDE CODE — NAVUUNA AI & AUTOMATION WORKSTREAM PROMPT

> Paste this prompt into Claude Code at the start of any session working on the AI/ML,
> RAG, agentic chatbot, or automation layers. It assumes the repo pair
> `nuvola-atlas-backend` (Laravel 11) and `nuvola-atlas-frontend` (React 18 + Vite 5),
> plus the FastAPI ingestion service. Companion docs: Navuuna Build Phases tracker,
> Backend Build Plan v1.1, tasks/todo.md.

---

## 1. WHO YOU ARE WORKING ON

You are working on **Navuuna Atlas** — a spatial intelligence platform for Nairobi
County. It computes a **Vitality Score** for 17 sub-counties from **13 indicators**
grouped into **4 equally-weighted pillars (0.25 each)**:

1. Social Wellbeing & Human Capital — healthcare_access, education_access, digital_connectivity
2. Safety & Security — crime_rates, emergency_response_access, disaster_exposure
3. Density & Scaling Dynamics — population_density, congestion, housing_pressure
4. Infrastructure & Environmental Safeguards — road_quality, energy_reliability, food_risk, waste_management

Data flows: **Daystar University (sole data partner, CSV/GeoJSON drops) → FastAPI
ingestion (Pydantic validation, data_cleaner.py WGS84/ISO-8601, anomaly_detector.py)
→ Laravel /ingest (X-Internal-Secret) → PostGIS on Supabase → async ScoreCalculator
job → Reverb WebSocket broadcast → live Mapbox map.**

## 2. AUTHORITY CHAIN & SESSION DISCIPLINE

- Read the **Build Phases tracker** before every session. Tracker > Backend Build Plan
  > this prompt. Where the codebase deviates from docs, the codebase wins and the
  deviation is logged.
- `tasks/todo.md` is the live tactical plan. Never delete or overwrite the tracker —
  only flip checkboxes `[ ]` → `[x]`, update `Last updated / HEAD` lines, and append.
- Do not redo completed `[x]` items. Do not skip incomplete items unless told to.
- **Plan Mode** for anything ≥ 3 steps or with architectural tradeoffs. Stop and ask
  on ambiguity.

## 3. NON-NEGOTIABLE STACK & RULES (GRANT-LOCKED)

- Laravel 11 (PHP 8.3+), Sanctum SPA tokens (8h TTL) + email 2FA, Gates/Policies only
  (no inline role checks), thin controllers → Service classes.
- Supabase PostgreSQL + PostGIS. Pooled :6543 for app traffic, direct :5432 for
  migrations. All migrations implement up() AND down(). Raw SQL banned except
  isolated, commented PostGIS spatial queries in a dedicated service/repository method.
```

```
- FastAPI (Python 3.13) ingestion service, isolated from the core platform;
  Python→PHP hop authenticated with X-Internal-Secret.
- **NO** Next.js, Rails, Django, MQTT, Go migrations, premature abstractions,
  hypothetical feature flags, or DB mocking in integration tests (Docker Postgres always).
- Never hardcode secrets (VITE_MAPBOX_ACCESS_TOKEN or anything else) — environment
  variables only. Never add `Co-Authored-By: Claude` to commits. Conventional Commits,
  per-slice, push only when green.
- ScoreCalculator is dispatched **only** as an async Laravel job — never from a controller.
- Nulls are excluded from score averages — never zero-biased. Null pillars render as '—'.
- **No vector RAG until unstructured documents are actually ingested.** Structured RAG
  via text-to-SQL (app/Services/Chat/) is the current pattern. Phase H below defines
  the trigger condition for lifting this rule — do not lift it early.

## 4. THE EXISTING AI SURFACE (DO NOT REBUILD — EXTEND)

The chat stack already ships behind the Vercel AI Gateway, mock-gated by
`USE_MOCK_CHAT` while billing is pending:

- `AiGatewayClient` — HTTP client for the Vercel AI Gateway.
- `ChatOrchestrator` — request lifecycle for one chat turn.
- `IntentRouter` — routes user intent (data question vs. general).
- `SchemaCatalog v2` — schema-aware context for SQL generation.
- `SqlGenerator` → `SqlGuard` → `SqlExecutor` — text-to-SQL with a hard safety gate
  (read-only, allow-listed tables, no DDL/DML).

Your job in the AI workstream is to evolve this pipeline into the layered
architecture below **without breaking the existing text-to-SQL path**, which remains
the default tool for structured questions.

## 5. TARGET AI ARCHITECTURE (PHASES G–J)

### Phase G — ML Layer (predictive intelligence)
Owner: FastAPI service (Python stays the ML runtime; Laravel never runs models).

1. **Vitality Forecaster** — time-series model over `zone_snapshots` (13 indicator
   columns per zone per snapshot_date). Start with gradient-boosted trees
   (LightGBM/XGBoost) or classical baselines (SARIMA/ETS) per indicator per zone;
   deep learning only when data volume justifies it. Output: 30/90-day projected
   pillar + composite scores with confidence intervals.
2. **anomaly_detector v2** — upgrade the shipped statistical spike-blocker to an
   ML detector (IsolationForest / seasonal-decomposition residuals) trained on
   accepted historical batches in `data_ingestion_logs` + `indicator_scores`.
   The statistical detector stays as the fallback — never remove it.
3. **Opportunity Ranker** — learning-to-rank (or transparent weighted heuristic v1)
```

```
   powering `/investor/opportunities`, tier-aware (basic|deal|sovereign).
4. **Data Quality Classifier** — scores each Daystar batch (completeness, drift,
   plausibility) and writes a quality grade onto `data_ingestion_logs`.

Serving: new FastAPI routes `/ml/forecast`, `/ml/rank`, `/ml/quality` — internal
only, X-Internal-Secret, called from Laravel Service classes (`Ml\ForecastService`
etc.). Models versioned in an `ml_models` registry table; artifacts in object
storage (R2/Vercel Blob per the Phase C decision). Every prediction row lands in
`ml_predictions` with model_version FK for auditability. Retraining is a scheduled
job with a champion/challenger gate — a new model only replaces `is_current` when it
beats the incumbent on a held-out window.

### Phase H — RAG Layer (grounded knowledge)
**Trigger condition (do not build before this is true):** unstructured documents
exist in the system — content_blocks/CMS copy, methodology docs, published reports,
the Daystar indicator spec, partner PDFs. Once true:

- Enable **pgvector** on Supabase (stays inside the grant-locked DB — no new vendor).
- `document_chunks` table: id, source_type, source_id, chunk_text, embedding
  vector(1536), metadata jsonb, embedded_at. HNSW index.
- Embedding pipeline as a Laravel job (`Rag\DocumentIndexer`) triggered on
  content_block save / report publish; embeddings generated through the **Vercel AI
  Gateway** (same gateway, no new keys in code).
- **Hybrid retrieval**: IntentRouter decides — structured/numeric question →
  text-to-SQL path (unchanged); narrative/methodology/"why" question → vector
  retrieval → grounded answer **with citations to source chunks**; mixed → both,
  merged by ChatOrchestrator.
- Guardrails: retrieval is role-scoped (firm-scoped reports only retrievable by that
  firm's investors — enforce via the same Gates/Policies + RLS, never in prompt text).
  Answers must cite chunk sources; if retrieval confidence is low, say so instead of
  guessing.

### Phase I — Agentic Chatbot & Background Agents
Evolve ChatOrchestrator from single-shot pipeline to a bounded **agent loop**
(plan → act → observe → repeat, max N steps, hard timeout):

- **Tool registry** (Laravel-side, each tool is a thin wrapper over an existing
  Service class so Gates/Policies apply automatically):
  `query_data` (SqlGuard-gated text-to-SQL), `search_docs` (RAG, Phase H),
  `get_zone`, `compare_zones`, `get_feed_status`, `export_report`
  (ZoneReportExporter), `manage_watchlist` (investor-only, firm.scope),
  `forecast_zone` (Phase G ML). Tools are **allow-listed per role** — the agent of a
  viewer literally cannot see investor tools.
- **Agent loop contract**: every step logged to `agent_runs` /
```

```
  `agent_steps` (input, tool, args, result hash, tokens, latency). Write-capable
  tools (watchlist, export) require an explicit user confirmation turn before
  execution. Budget guard: per-conversation token + tool-call ceiling; the loop
  degrades to a plain answer when exceeded.
- **Prompt-injection defense**: retrieved chunks and SQL results are data, never
  instructions — wrap them in delimited data blocks; SqlGuard and the tool
  allow-list are the enforcement layer, not the prompt.
- **Background agents** (scheduled, not chat-driven — each one is a Laravel job
  chain that may call the LLM through AiGatewayClient):
  - *Feed Watchdog Agent* — reads FeedStatusService, detects staleness patterns,
    drafts a diagnosis + Slack alert (via automation layer) instead of a bare threshold ping.
  - *Data Quality Agent* — reviews flagged batches (Phase G classifier), summarizes
    what's wrong per batch for the admin console.
  - *Investor Digest Agent* — weekly per-firm digest: watchlist score movements,
    new opportunities, one-paragraph narrative; renders through ZoneReportExporter.
  - *Ops Reporter Agent* — Monday morning internal digest: ingestion volume, error
    rates from Sentry, pipeline health.

### Phase J — Automations (n8n + Laravel-native)
**Decision rule:** anything touching app data or requiring app permissions is a
Laravel Scheduler/Horizon job (inside the trust boundary). Anything that is
cross-service glue (email, Slack, third-party SaaS, human-in-the-loop approvals) is
an **n8n workflow** (self-hosted on the same DigitalOcean box or Fly.io app,
behind Cloudflare Access). n8n talks to Navuuna **only** through the public
/api/v1/* surface with a dedicated `automation` API token (rate-limit override via
personal_access_tokens) — never direct DB access.

Priority workflows:
1. **Daystar drop intake** — n8n watches the drop channel (email attachment /
   shared-drive webhook) → posts file to FastAPI /ingest → posts validation summary
   to Slack #data-feeds. Removes the manual hand-off that Phase B is blocked on.
2. **Feed staleness escalation** — Laravel detects (FeedStatusService), n8n routes:
   warn Slack at T+expected_frequency, email Joy at 2×, create tracker task at 3×.
3. **Weekly investor digest dispatch** — Laravel renders (Investor Digest Agent),
   n8n handles delivery, bounces, unsubscribes.
4. **Backup + restore-drill verification** — n8n triggers monthly restore drill,
   runs checksum comparison, posts pass/fail evidence to Slack.
5. **Sentry → triage** — n8n receives Sentry webhooks, deduplicates, applies a
   severity rubric, opens tracker items for new error classes only.
6. **Model retraining pipeline** — cron in n8n calls FastAPI retrain endpoint,
   waits on champion/challenger result, requests human approval in Slack before
   flipping `is_current` in ml_models.

## 6. NEW TABLES YOU MAY CREATE (dependency order)
```

```
1. `conversations` — id uuid PK, user_id FK, title, created_at
2. `conversation_messages` — conversation_id FK, role enum, content, tool_calls jsonb, tokens
3. `ml_models` — id, name, version, task enum(forecast|anomaly|rank|quality),
   metrics jsonb, artifact_uri, is_current partial unique idx, trained_at
4. `ml_predictions` — model_id FK, zone_id FK, horizon, payload jsonb, created_at
5. `agent_runs` — id uuid, agent enum, trigger enum(chat|schedule), user_id nullable FK,
   status, token_cost, started_at, ended_at
6. `agent_steps` — agent_run_id FK, seq, tool, args jsonb, result_hash, latency_ms
7. `automation_runs` — workflow_key, external_id (n8n execution id), status, payload_hash, ran_at
8. `document_chunks` (Phase H only) — as specified above, pgvector.

All reversible, all through `artisan migrate`, RLS reviewed per table.

## 7. DEFINITION OF DONE (unchanged + additions)

Run after every meaningful slice:
1. `cd nuvola-atlas-frontend && npx tsc --noEmit`
2. `cd nuvola-atlas-frontend && npx vite build`
3. `cd nuvola-atlas-frontend && npx vitest run`
4. `cd nuvola-atlas-backend && php artisan route:list --path=api`
5. `cd nuvola-atlas-backend && php vendor/phpunit/phpunit/phpunit --no-coverage`
   (Docker Postgres up — never mocked)

Additions for this workstream:
6. `cd ingestion && ruff check . && mypy . && pytest` (ruff+mypy are an open Phase B item — add the config if absent)
7. Every new agent tool has: a Policy test, a SqlGuard/allow-list test, and an
   agent_steps logging assertion.
8. Every ML endpoint has a contract test with a frozen model fixture (deterministic).

## 8. HOW TO BEHAVE

- Extend, don't rewrite. The text-to-SQL path, ScoreCalculator, and the ingestion
  scaffold are shipped and tested — new AI capability wraps around them.
- Cost discipline: every LLM/ML call path needs a budget guard (the Phase C spend-guard
  pattern applies to the AI Gateway too). USE_MOCK_CHAT-style gates for anything
  whose billing isn't confirmed.
- When Daystar data is absent (Phase B blocker), build against seeded fixtures but
  keep the seam explicit — one env flag, no scattered conditionals.
- Update the tracker checkbox the moment a task lands. Log every documented deviation.

**First action of every session:** read the tracker, read tasks/todo.md, state which
phase and which unchecked items you are about to work, and enter Plan Mode if the
slice is ≥ 3 steps.
```

End of Backend Build Plan v1.1 — Navuuna Technical Series.